

IREE 中的 GPUMultiBuffering pass 详解

《AI 编译器开发指南》一书 5.2.2 节在介绍 CUTLASS 中的张量核编程方法时提到，为了更好地将内存访问延迟被隐藏在计算过程中，提高并行度，CUTLASS 采用了软件流水线机制，将计算的主循环分为软件可配置的若干个流水线阶段。上游流水线阶段在加载数据的同时，下游流水线阶段可执行计算，计算使用的操作数来自前一上游流水线阶段的加载操作。软件流水线结构如下图所示：

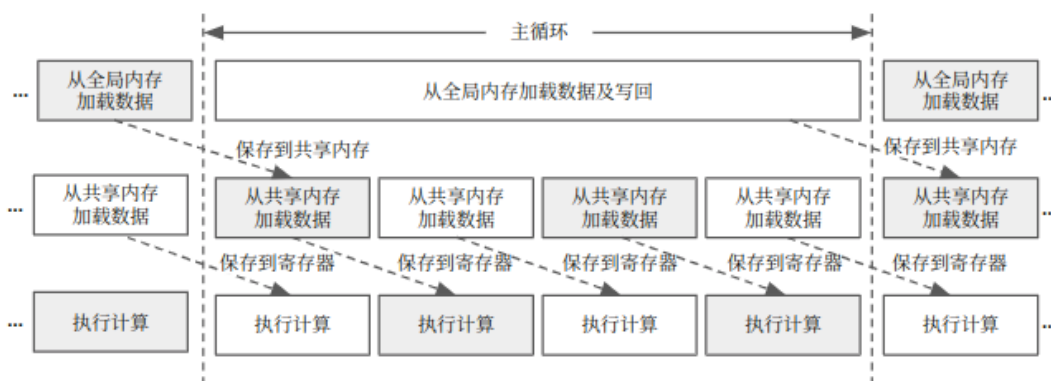


图 1. 软件流水线结构

通过广泛使用软件流水线技术，CUTLASS 可以最大程度地利用 GPU 的计算资源，并提高矩阵乘法的性能。由上图可见，软件流水线阶段包括：

1. 从全局内存加载数据
2. 将数据拷贝至共享内存 / 进行必要的预处理
3. 执行计算
4. 将计算结果写回共享内存或全局内存

在软件流水线结构中，GPU 在使用共享内存的数据执行当前计算的同时，还应从全局内存中读取下一次计算所需数据。因此，从存储层次结构的角度看，在共享内存级别应使用多缓冲模式，保证在上游流水线阶段将数据写入共享内存的同时，下游流水线阶段可以从共享内存中加载数据到寄存器。换言之，循环中的不同迭代使用不同缓冲区，从而消除迭代间的数据依赖。多缓冲模式中应使用的缓冲数量取决于流水线深度（也称流水线阶段数量）。

MLIR 中的 GPUMultiBuffering pass 是实现软件流水线的重要组成部分。其核心思想是通过引入多缓冲，使得循环迭代尽可能并行执行，从而为最终实现软件流水线创造条件。

GPUMultiBuffering pass 的入口函数是 GPUMultiBufferingPass::runOnOperation()，其目的是在 MLIR 函数中对共享内存分配操作执行多缓冲 (multi-buffering) 优化，代码如下所示：

```
struct GPUMultiBufferingPass final
: impl::GPUMultiBufferingPassBase<GPUMultiBufferingPass> {
using GPUMultiBufferingPassBase::GPUMultiBufferingPassBase;

void runOnOperation() override {
    FunctionOpInterface funcOp = getOperation();

    // First hoist all shared memory allocations to the entry block of the
    // function. We can see memref.alloc in loops after buffering scf.forall
```

```

// with promoted shared memory usage inside.

SmallVector<memref::AllocOp> allocs;
// Collect all the alloc operations.
funcOp.walk([&](memref::AllocOp allocOp) {
    if (hasSharedMemoryAddressSpace(allocOp.getType()))
        allocs.push_back(allocOp);
});

assert(funcOp.getBlocks().size() == 1);
for (memref::AllocOp allocOp : allocs) {
    if (allocOp->getParentOp() != funcOp)
        allocOp->moveBefore(&*funcOp.begin()->begin());
}

// Then perform multibuffering transformations.

allocs.clear();
// Collect all the alloc operations.
funcOp.walk([&](memref::AllocOp allocOp) {
    // Skip allocations not used in a loop.
    for (Operation *user : allocOp->getUsers()) {
        auto loop = user->getParentOfType<scf::ForOp>();
        if (!loop)
            return WalkResult::advance();
    }
    allocs.push_back(allocOp);
    return WalkResult::advance();
});
// Apply multi-buffering to all of them.
for (memref::AllocOp alloc : allocs) {
    if (failed(memref::multiBuffer(alloc, numBuffers))) {
        // Error out and stop if any buffer cannot be multi buffered, as future
        // software pipelining transformations will assume this happened.
        alloc.emitOpError("cannot be multi-buffered");
        return signalPassFailure();
    }
}
};

```

根据上述代码可总结得到 GPU MultiBufferingPass::runOnOperation() 函数流程图如下图所示：

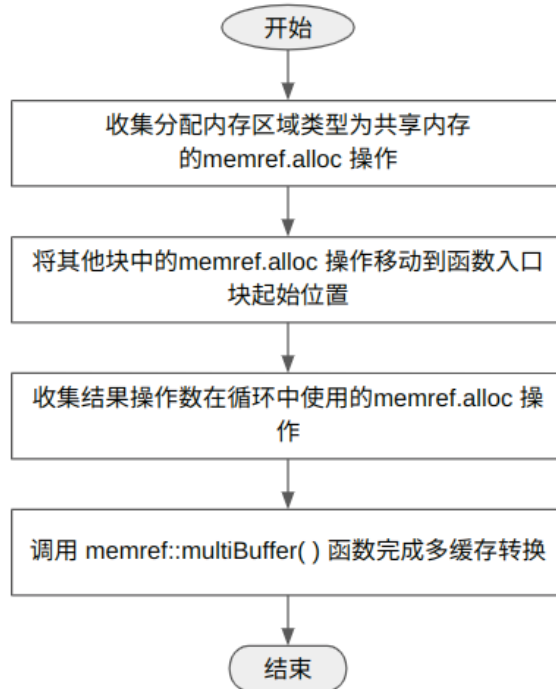


图 2. GPU MultiBuffering Pass::runOnOperation() 函数流程图

runOnOperation() 函数首先遍历 IR 函数中的所有 memref.alloc 操作。如果根据 hasSharedMemoryAddressSpace() 函数判断 memref.alloc 操作分配的内存区域类型(MemRefType)属于共享内存地址空间, 则将该 memref.alloc 操作保存在分配操作集合 allocs 中。

然后, runOnOperation() 函数遍历分配操作集合 allocs 中的所有 memref.alloc 操作, 检查 memref.alloc 操作的父操作是否为函数操作 funcOp。如果不是, 意味着该 memref.alloc 操作在函数操作内部的某个块中, 于是将这类 memref.alloc 操作移动到函数入口块(entry block)起始位置, 确保内存分配在循环之前完成, 为后续的多缓冲转换做好准备。。

接下来, runOnOperation() 函数执行多缓冲转换。在执行多缓冲转换之前, 需要先重新收集分配操作集合, 条件是 memref.alloc 操作结果的使用者在 scf.for 操作内部, 即, GPU MultiBuffering pass 只对结果操作数在循环中使用的、内存区域类型为共享内存的 memref.alloc 操作进行多缓冲转换。

例如, 以下示例代码中的 memref.alloc 操作都在函数入口块起始位置, 分配的内存存在共享内容存空间且操作的使用者在 scf.for 操作内部, 符合上述多缓冲转换条件, 于是可将 memref.alloc 操作保存在分配操作集合 allocs 中。

```

%0 = memref.alloc() : memref<4x128xf32, #gpu.address_space<workgroup>>
scf.for %iv = %c1 to %c1024 step %c3 {
  memref.copy %1, %0 : memref<4x128xf32> to memref<4x128xf32,
#gpu.address_space<workgroup>>
  "some_use"(%0) : (memref<4x128xf32, #gpu.address_space<workgroup>>) -> ()
}
  
```

实际的多缓冲转换功能在 `memref::multiBuffer()` 函数中实现。`runOnOperation()` 函数在调用该函数时,除了函数参数中指定需要进行多缓冲的 `memref.alloc` 操作外,还要指定缓冲区数量(`numBuffers`),即流水线深度。流水线深度由 `getTensorCoreConfig()` 函数在确定张量核操作应采用的最优工作组分块大小和工作组大小配置时指定。`getTensorCoreConfig()` 函数的详细功能介绍见《MLIR 开发指南》6.2.1 节。

`memref::multiBuffer()` 函数的作用是消除连续循环迭代之间对临时分配缓冲区的依赖,并通过新生成的 `memref.alloc` 操作分配多个缓冲区,使得每次迭代可以使用不同的缓冲区,从而实现并行化或流水线执行。

在以上示例代码中, `memref.alloc` 操作分配了大小为 `4x128` 的 `memref %0`。在 `scf.for` 循环中, `memref.copy` 操作将 `%1` 中的数据复制到 `%0` 中,并以 `%0` 作为 "some_use" 操作的源操作数。由于每次迭代都使用相同的 `%0`,因此存在数据依赖。

如果指定流水线深度为 5,经过多缓冲转换后的,应生成如下代码:

```
%0 = memref.alloc() : memref<5x4x128xf32, #gpu.address_space<workgroup>>
scf.for %iv = %c1 to %c1024 step %c3 {
  %s = arith.subi %iv, %c1 : index
  %d = arith.divsi %s, %c3 : index
  %i = arith.remsi %d, %c5 : index
  %sv = memref.subview %0[%i, 0, 0] [1, 4, 128] [1, 1, 1] :
    memref<5x4x128xf32> to memref<4x128xf32, strided<[128, 1], offset: ?>>
  memref.copy %1, %sv : memref<4x128xf32> to memref<4x128xf32>, ,
#gpu.address_space<workgroup>>
  "some_use"(%sv) : (memref<4x128xf32, strided<...>) -> ()
}
```

新的 `memref.alloc` 操作分配了大小为 `5x4x128`、5 倍于原缓冲区大小的多个缓冲区。`scf.for` 循环的 $341 = (1024 - 1) / 3$ 次迭代平均分配使用不同的缓冲区。为了将 5 个缓冲区均匀分配给 341 次迭代,通过计算公式 $i = (iv - 1) / 3 \% 5$ 可得缓冲区索引值 i 为 0 到 4。上述代码中的 `%d` 表示迭代索引,值为 $[0, 340]$ 。`%i` 为计算得到的缓冲区索引。

迭代索引、缓冲区索引以及缓冲区实际位置的对应关系如下图所示:

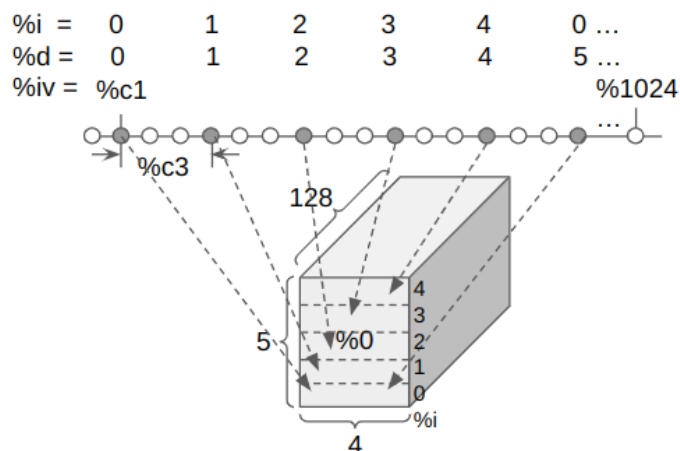


图 3. 迭代索引、缓冲区索引以及缓冲区实际位置的对应关系

通过将迭代索引取模映射到缓冲区索引，使得不同迭代可以使用不同的缓冲区，消除了迭代间的数据依赖。

上述代码中的 `memref.subview` 操作从多个缓冲区中生成了一个大小与原始大小 (4x128) 相同的子视图 `%sv`，该子视图为多个缓冲区 `%0` 的一个切片(即一个缓冲区)，偏移位置由缓冲区索引 `%i` 指定。`memref.copy` 操作将 `%1` 中的数据复制到 `%sv` 中，并以 `%sv` 作为 "some_use" 操作的源操作数。

下图显示了 `memref.subview` 操作生成的切片与多个缓冲区的关系。

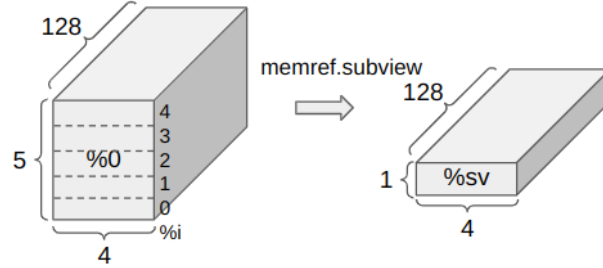


图 4. `memref.subview` 操作生成的切片与多个缓冲区的关系

本文使用如下测试用例作为输入 IR，辅助分析多缓冲转换工作流程。

```
func.func @matmul_dispatch_0_matmul_512x512x512_f16() {
  %alloc = memref.alloc() : memref<64x64xf16, #gpu.address_space<workgroup>>
  %alloc_0 = memref.alloc() : memref<64x64xf16, #gpu.address_space<workgroup>>
  ...
  scf.forall (%arg0, %arg1) = (0, 0) to (512, 512) step (64, 64) {
  ...
    scf.for %arg2 = %c0 to %c512 step %c64 {
    ...
      memref.copy %subview_4, %alloc_0 {...} : memref<64x64xf16, ...>, ...> to
      memref<64x64xf16, #gpu.address_space<workgroup>>
      memref.copy %subview_5, %alloc {...} : memref<64x64xf16, ...>, ...> to
      memref<64x64xf16, #gpu.address_space<workgroup>>
      ...
      %subview_8 = memref.subview %alloc_0[%5, 0] [32, 64] [1, 1] : memref<64x64xf16,
      #gpu.address_space<workgroup>> to memref<32x64xf16, strided<[64, 1], offset: ?>,
      #gpu.address_space<workgroup>>
      %subview_9 = memref.subview %alloc[0, %6] [64, 32] [1, 1] : memref<64x64xf16,
      #gpu.address_space<workgroup>> to memref<64x32xf16, strided<[64, 1], offset: ?>,
      #gpu.address_space<workgroup>>
      ...
      linalg.matmul {...} ins(%subview_8, %subview_9 ...
      ...
    }
  }
}
```

上述测试用例中的 memref.alloc 操作都在函数入口块起始位置, 分配的内存存在共享内容存储空间且操作的使用者在 scf.for 操作内部, 符合上述多缓冲转换条件。

上述测试用例中的 scf.for 的下限为 0, 上限为 512, 步长为 64, 因此循环共有 $8 = (512 - 0) / 64$ 次迭代。流水线深度设置为 4。

memref::multiBuffer() 函数代码实现如下:

```
FailureOr<memref::AllocOp>
mlir::memref::multiBuffer(RewriterBase &rewriter, memref::AllocOp allocOp,
                          unsigned multiBufferingFactor, bool skipOverrideAnalysis) {
    DominanceInfo dom(allocOp->getParentOp());
    LoopLikeOpInterface candidateLoop;
    for (Operation *user : allocOp->getUsers()) {
        auto parentLoop = user->getParentOfType<LoopLikeOpInterface>();
        if (!parentLoop) {
            if (isa<memref::DeallocOp>(user)) {
                continue;
            }
            return failure();
        }
        if (!skipOverrideAnalysis) {
            if (!overrideBuffer(user, allocOp.getResult())) {
                continue;
            }
            if (llvm::any_of(allocOp->getUsers(), [&](Operation *otherUser) {
                return !dom.dominates(user, otherUser);
            })) {
                continue;
            }
        } else {
            if (llvm::any_of(allocOp->getUsers(), [&](Operation *otherUser) {
                return !isa<memref::DeallocOp>(otherUser) &&
                    !parentLoop->isProperAncestor(otherUser);
            })) {
                continue;
            }
        }
        candidateLoop = parentLoop;
        break;
    }

    if (!candidateLoop) {
        return failure();
    }
}
```

```

std::optional<Value> inductionVar = candidateLoop.getSingleInductionVar();
std::optional<OpFoldResult> lowerBound = candidateLoop.getSingleLowerBound();
std::optional<OpFoldResult> singleStep = candidateLoop.getSingleStep();
if (!inductionVar || !lowerBound || !singleStep ||
    !llvm::hasSingleElement(candidateLoop.getLoopRegions())) {
    return failure();
}

if (!dom.dominates(allocOp.getOperation(), candidateLoop)) {
    return failure();
}

// 1. Construct the multi-buffered memref type.
ArrayRef<int64_t> originalShape = allocOp.getType().getShape();
SmallVector<int64_t, 4> multiBufferedShape{multiBufferingFactor};
llvm::append_range(multiBufferedShape, originalShape);
MemRefType mbMemRefType = MemRefType::Builder(allocOp.getType())
    .setShape(multiBufferedShape)
    .setLayout(MemRefLayoutAttrInterface());

// 2. Create the multi-buffered alloc.
Location loc = allocOp->getLoc();
OpBuilder::InsertionGuard g(rewriter);
rewriter.setInsertionPoint(allocOp);
auto mbAlloc = rewriter.create<memref::AllocOp>(
    loc, mbMemRefType, ValueRange{}, allocOp->getAttrs());

// 3. Within the loop, build the modular leading index (i.e. each loop
// iteration %iv accesses slice ((%iv - %lb) / %step) % %mb_factor).
rewriter.setInsertionPointToStart(&candidateLoop.getLoopRegions().front()->front());
Value ivVal = *inductionVar;
Value lbVal = getValueOrCreateConstantIndexOp(rewriter, loc, *lowerBound);
Value stepVal = getValueOrCreateConstantIndexOp(rewriter, loc, *singleStep);
AffineExpr iv, lb, step;
bindDims(rewriter.getContext(), iv, lb, step);
Value bufferIndex = affine::makeComposedAffineApply(
    rewriter, loc, ((iv - lb).floorDiv(step)) % multiBufferingFactor,
    {ivVal, lbVal, stepVal});

// 4. Build the subview accessing the particular slice, taking modular
// rotation into account.
int64_t mbMemRefTypeRank = mbMemRefType.getRank();
IntegerAttr zero = rewriter.getIndexAttr(0);
IntegerAttr one = rewriter.getIndexAttr(1);

```

```

SmallVector<OpFoldResult> offsets(mbMemRefTypeRank, zero);
SmallVector<OpFoldResult> sizes(mbMemRefTypeRank, one);
SmallVector<OpFoldResult> strides(mbMemRefTypeRank, one);
// Offset is [bufferIndex, 0 ... 0 ].
offsets.front() = bufferIndex;
// Sizes is [1, original_size_0 ... original_size_n ].
for (int64_t i = 0, e = originalShape.size(); i != e; ++i)
    sizes[1 + i] = rewriter.getIndexAttr(originalShape[i]);
// Strides is [1, 1 ... 1 ].
auto dstMemref =
    cast<MemRefType>(memref::SubViewOp::inferRankReducedResultType(
        originalShape, mbMemRefType, offsets, sizes, strides));
Value subview = rewriter.create<memref::SubViewOp>(loc, dstMemref, mbAlloc,
    offsets, sizes, strides);

// 5. Due to the recursive nature of replaceUsesAndPropagateType , we need to
// handle dealloc uses separately..
for (OpOperand &use : llvm::make_early_inc_range(allocOp->getUses())) {
    auto deallocOp = dyn_cast<memref::DeallocOp>(use.getOwner());
    if (!deallocOp) continue;
    OpBuilder::InsertionGuard g(rewriter);
    rewriter.setInsertionPoint(deallocOp);
    auto newDeallocOp = rewriter.create<memref::DeallocOp>(deallocOp->getLoc(), mbAlloc);
    (void)newDeallocOp;
    rewriter.eraseOp(deallocOp);
}

// 6. RAUW with the particular slice, taking modular rotation into account.
replaceUsesAndPropagateType(rewriter, allocOp, subview);

// 7. Finally, erase the old allocOp.
rewriter.eraseOp(allocOp);

return mbAlloc;
}

```

memref::multiBuffer() 函数首先遍历 allocOp 的所有使用者，确保 allocOp 使用者的父操作是 LoopLikeOpInterface 的实例，即，确保使用者在循环内。数据结构 LoopLikeOpInterface 是在 LoopLikeInterface.td 中定义的操作接口，用于描述和处理具有循环行为的操作，如 scf.for 和 scf.while 等循环操作。

如果 allocOp 的某一个使用者在循环内，则调用 overrideBuffer() 函数判断 allocOp 操作结果操作数的使用者是否为 memref.copy 操作，且 allocOp 操作结果操作数是否为 memref.copy 操作的目的操作数。如果是，则认为前一次迭代的数据被 memref.copy 操作完全覆盖，因此不存在循环携带依赖；否则，如果 allocOp 操作的使用者不是 memref.copy 操作，或 allocOp 操作结果操

作数不是 memref.copy 操作的目的操作数, 则认为前一次迭代的数据仍然在后续迭代中使用, 因此可能存在循环携带依赖。memref::multiBuffer() 函数只对不存在循环携带依赖的分配操作执行多缓冲转换。

例如, 在上述示例代码中, memref.alloc 操作的使用者为循环内的 memref.copy 操作, 且 memref.alloc 操作的结果 %0 是该 memref.copy 操作的目的操作数, 为前一次迭代产生的 %0 数据被 memref.copy 操作完全覆盖, 因此不存在循环携带依赖, 可以继续执行多缓冲转换。

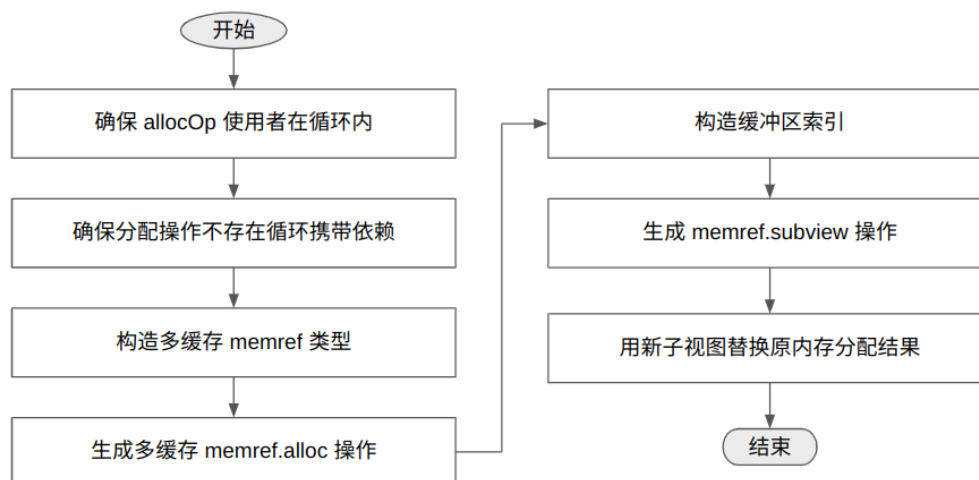


图 5. memref::multiBuffer() 函数执行流程

memref::multiBuffer() 函数实现多缓冲转换的代码功能可分为以下几个部分。

(1) 构造多缓冲 memref 类型

多缓冲转换通过增加额外维度来创建多缓冲区, 必然导致 memref 类型的变化。为此, memref::multiBuffer() 函数的第一部分功能是根据原始 memref.alloc 操作的类型和形状, 结合指定的流水线深度, 构造新的多缓冲 memref 类型。

例如, 多缓冲转换前, 示例代码中的 memref.alloc 操作为:

```
%alloc = memref.alloc() : memref<64x64xf16, #gpu.address_space<workgroup>>
%alloc_0 = memref.alloc() : memref<64x64xf16, #gpu.address_space<workgroup>>
```

其原始形状 originalShape 为 [64, 64]。结合指定的流水线深度 4, 可以构造新的多缓冲形状数组 multiBufferedShape 为 [4, 64, 64], 即, multiBufferedShape 数组中新增加的维度 0 的大小为流水线深度 4。以 multiBufferedShape 为参数, 调用 MemRefType::Builder() 接口构造的多缓冲 memref 类型 mbMemRefType 为:

```
memref<4x64x64xf16, #gpu.address_space<workgroup>>
```

(2) 生成多缓冲 memref.alloc 操作

memref::multiBuffer() 函数通过调用 rewriter.create<memref::AllocOp>() 接口生成多缓冲 memref.alloc 操作, 分配多缓冲内存区域。其中, 接口参数 mbMemRefType 指定了新分配的内存区域的类型。该类型具有额外的维度, 用于索引不同的缓冲区, 维度大小为流水线深度。

针对上述示例代码, memref::multiBuffer() 函数生成的多缓冲 memref.alloc 操作为:

```
%alloc = memref.alloc() : memref<4x64x64xf16, #gpu.address_space<workgroup>>
%alloc_0 = memref.alloc() : memref<4x64x64xf16, #gpu.address_space<workgroup>>
```

(3) 构造缓冲区索引

如前文所述, 为了将多缓冲中的各缓冲区均匀分配给各循环迭代, 需要通过以下公式计算不同循环迭代对应的缓冲区索引:

$$\text{bufer_index} = (\text{iv} - \text{lb}) / \text{step} \% \text{pipeline_depth}$$

其中, iv(indction variable)为循环归纳变量, lb(lower bound)为循环下限, step 为循环步长。为了计算上述缓冲区索引, memref::multiBuffer() 函数调用 bindDims() 接口将 AffineExpr 变量 iv、lb、step 绑定到从 0 开始的对应维度表达式。然后, 通过 affine::makeComposedAffineApply() 接口构造仿射表达式 ((iv - lb).floorDiv(step)) % multiBufferingFactor。针对上述示例代码, memref::multiBuffer() 函数构造的缓冲区索引为:

```
%5 = affine.apply affine_map<(d0) -> ((d0 floordiv 64) mod 4)>(%arg2)
```

缓冲区索引 %5 的计算方式与前述示例代码中的 %i 相同:

```
%s = arith.subi %iv, %c1 : index
%d = arith.divsi %s, %c3 : index
%i = arith.remsi %d, %c5 : index
```

(4) 生成 memref.subview 操作

在获得缓冲区索引后, memref::multiBuffer() 函数根据缓冲区索引调用 rewriter.create<memref::SubViewOp>() 接口, 生成 memref.subview 操作, 从多个缓冲区中生成了一个大小与原始大小 (4x128) 相同的子视图。该 memref.subview 操作的偏移量是上述计算得到的缓冲区索引 %5, 大小和步长根据 allocOp 的原始形状推断得到。针对上述示例代码, memref::multiBuffer() 函数生成 memref.subview 操作为:

```
%subview_5 = memref.subview %alloc[%5, 0, 0] [1, 64, 64] [1, 1, 1] : memref<4x64x64xf16,
#gpu.address_space<workgroup>> to memref<64x64xf16, strided<[64, 1], offset: ?>,
#gpu.address_space<workgroup>>
```

(5) 处理 memref.dealloc 操作

由于本文测试用例不涉及 memref.dealloc 操作, 略。

(6) 用新子视图替换原内存分配结果

上述“生成 memref.subview 操作”过程, 根据迭代次数和缓冲区索引生成了与之对应子视图 %subview_5。%subview_5 的大小与原始 %alloc 大小相同。memref::multiBuffer() 函数调用 replaceUsesAndPropagateType() 函数, 将原始 IR 中所有以 %alloc 作为操作数的使用都应替换为 %subview_5, 并处理由于该替换引起的其他操作的连锁反应。replaceUsesAndPropagateType() 函数代码实现如下:

```

static void replaceUsesAndPropagateType(RewriterBase &rewriter,
                                       Operation *oldOp, Value val) {
    SmallVector<Operation *> opsToDelete;
    SmallVector<OpOperand *> operandsToReplace;
    for (OpOperand &use : oldOp->getUses()) {
        auto subviewUse = dyn_cast<memref::SubViewOp>(use.getOwner());
        if (!subviewUse) {
            operandsToReplace.push_back(&use);
            continue;
        }
        OpBuilder::InsertionGuard g(rewriter);
        rewriter.setInsertionPoint(subviewUse);
        Type newType = memref::SubViewOp::inferRankReducedResultType(
            subviewUse.getType().getShape(), cast<MemRefType>(val.getType()),
            subviewUse.getStaticOffsets(), subviewUse.getStaticSizes(),
            subviewUse.getStaticStrides());
        Value newSubview = rewriter.create<memref::SubViewOp>(
            subviewUse->getLoc(), cast<MemRefType>(newType), val,
            subviewUse.getMixedOffsets(), subviewUse.getMixedSizes(),
            subviewUse.getMixedStrides());
        replaceUsesAndPropagateType(rewriter, subviewUse, newSubview);
        opsToDelete.push_back(use.getOwner());
    }
    for (OpOperand *operand : operandsToReplace) {
        Operation *op = operand->getOwner();
        rewriter.startOpModification(op);
        operand->set(val);
        rewriter.finalizeOpModification(op);
    }
    for (Operation *op : opsToDelete)
        rewriter.eraseOp(op);
}

```

replaceUsesAndPropagateType() 函数的功能是用新的值 (由函数参数 val 指定) 替换原始 IR 中原始操作 (由函数参数 oldOp 指定) 的使用。原始 IR 中的原始操作可以是 memref.alloc 操作, 也可以是 memref.copy 或 linalg.matmul 等其他类型操作。例如, 在测试用例中, 原始 memref.alloc 操作的结果为原始 %alloc:

```
%alloc = memref.alloc() : memref<64x64xf16, #gpu.address_space<workgroup>>
```

原始 %alloc 的使用者是 scf.for 循环中的 memref.copy 和 memref.subview 操作。在上述“生成 memref.subview 操作”过程中生成了与迭代次数和缓冲区索引对应的新的子视图 %subview_5, 因此, 应该将原始 %alloc 使用者的使用替换为 val 参数指定的新子视图, 即, 将原始 memref.copy 和 memref.subview 操作的操作数 %alloc 替换为 %subview_5。

replaceUsesAndPropagateType() 函数首先处理使用者中的 memref.subview 操作。%alloc 的使用者 memref.subview 操作为：

```
%subview_9 = memref.subview %alloc[0, %6] [64, 32] [1, 1] : memref<64x64xf16,
#gpu.address_space<workgroup>> to memref<64x32xf16, strided<[64, 1], offset: ?>,
#gpu.address_space<workgroup>>
```

为了将 %alloc 替换为 %subview_5, replaceUsesAndPropagateType() 函数生成新的 memref.subview 操作, 并调用 inferRankReducedResultType() 函数根据新子视图 val 的类型推断新 memref.subview 操作的类型。将 %alloc 替换为 %subview_5 后的新 memref.subview 操作为：

```
%subview_11 = memref.subview %subview_5[0, %7] [64, 32] [1, 1] : memref<64x64xf16,
strided<[64, 1], offset: ?>, #gpu.address_space<workgroup>> to memref<64x32xf16,
strided<[64, 1], offset: ?>, #gpu.address_space<workgroup>>
```

新 memref.subview 操作的操作数已经从 %alloc 替换为 %subview_5。由于子视图操作的类型依赖于其输入的操作数类型, 简单的 replaceAllUses 不足以处理这种情况, 需要进行类型传播并删除旧的子视图操作。因此, 将原 memref.subview 操作被放入待删除操作集合 opsToDelete 中。上述 %alloc 到 %subview_5 的替换将引起其他相关操作的更新。例如, 测试用例中的原始 memref.copy 操作也应将其操作数 %alloc 替换为 %subview_5。原 memref.subview 操作删除将导致以其结果为输入操作数的原始 linalg.matmul 操作需要更新。这类需要替换的非 memref.subview 操作输入操作数保存在待替换操作集合 operandsToReplace 中, 并使用延迟替换和删除则保证代码的正确性。

下图总结了 replaceUsesAndPropagateType() 函数针对测试用例的内存分配结果替换过程。

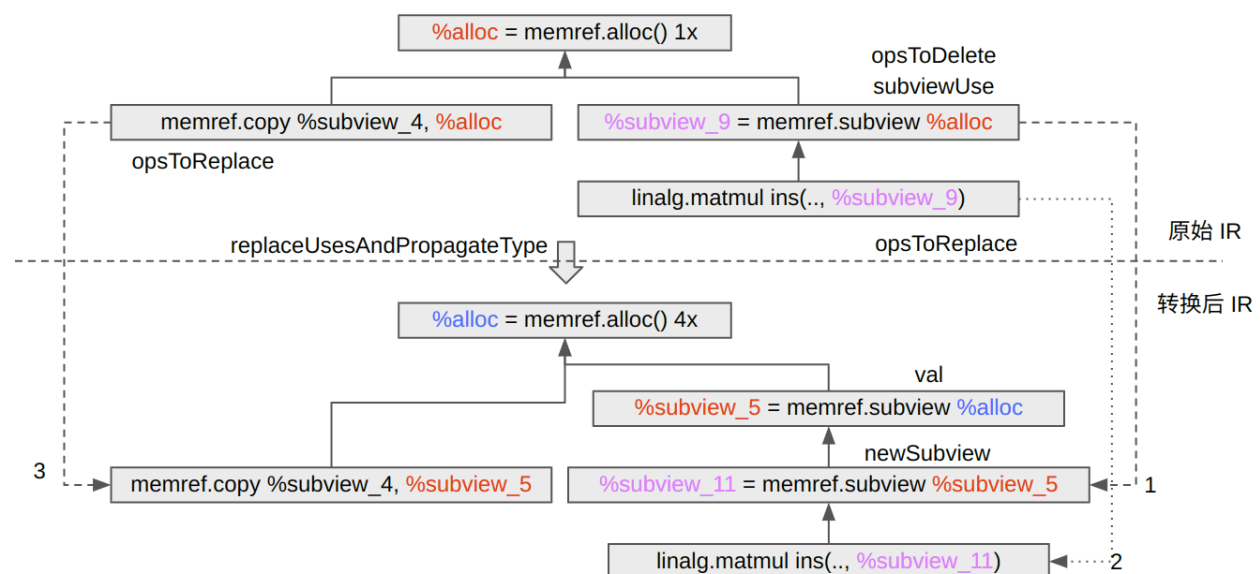


图 6. 内存分配结果替换过程

比较 pipeline depth = 1 和 pipeline depth = 4 时的 CUDA 后端输出 LLVM IR 可以看到, pipeline depth 决定了在进行全局内存到共享内存异步拷贝 (cp.async) 时, 能够同时“排队”或“并行”执行的拷贝批次数, 即, 当 pipeline depth = 1 时, 只允许在任意时刻进行一次异步拷贝, 拷贝完成后再进行下一次拷贝, 代码实现较为简单, 流水化程度最低; 当 pipeline depth = 4 时, 允许在任意时刻同时最多有四次拷贝正在进行, 拷贝与计算更好地重叠, 能进一步隐藏全局内存访问的延迟, 带来更高并行度和潜在性能提升。但相应地, LLVM IR 也会更复杂, 需要多路共享内存缓冲来存放这些正在拷贝的数据。

当 pipeline depth = 1 时, LLVM IR 中通常只看到一个共享内存全局变量声明, 如:

```
@__shared_memory__ = private addrspace(3) global [4096 x i8] undef, align 1
```

整个复制-计算过程只用这一个共享内存缓冲区保存数据。

当 pipeline depth = 4 时, LLVM IR 中出现多个共享内存全局变量声明, 如:

```
@__dynamic_shared_memory__ = external addrspace(3) global [0 x i8], align 16
```

```
@__shared_memory__1 = private addrspace(3) global [32 x [32 x half]] undef, align 2
```

```
@__shared_memory__0 = private addrspace(3) global [4 x [32 x [32 x half]]] undef, align 2
```

```
@__shared_memory__ = private addrspace(3) global [4 x [32 x [32 x half]]] undef, align 2
```

当 pipeline depth = 4 时, 共享内存全局变量的容量比 pipeline depth = 1 时共享内存全局变量的容量更大, 但不是 4 倍关系。

为什么 iluvatar backend 生成的 LLVM IR 中的, 当 pipeline depth = 1 时, 共享内存全局变量声明为:

```
@__shared_memory__0 = private addrspace(3) global [64 x [64 x half]] undef, align 2
```

```
@__shared_memory__ = private addrspace(3) global [64 x [64 x half]] undef, align 2
```

当 pipeline depth = 4 时, 共享内存全局变量声明为:

```
@__shared_memory__0 = private addrspace(3) global [4 x [64 x [64 x half]]] undef, align 2
```

```
@__shared_memory__ = private addrspace(3) global [4 x [64 x [64 x half]]] undef, align 2
```

当 pipeline depth = 4 时, 共享内存全局变量的容量与 pipeline depth = 1 时共享内存全局变量的容量正好是 4 倍关系。